Pushdown Automata

Finite Control with an Unbounded Stack

Sheikh Azizul Hakim

Theory of Computation

Department of Computer Science and Engineering, BUET

Where We Are

The Pumping Lemma showed that some languages need more memory than a finite automaton possesses.

Today's question

What is the smallest useful extension of a finite automaton that can remember an unbounded amount of structured information?

Learning Goals

By the end of this lecture, you should be able to:

1. explain why a stack enables nonregular behavior;
2. describe a PDA informally and formally;
3. read and write PDA transition labels;
4. trace a PDA computation using configurations;
5. design PDAs for equality and palindrome-style languages;
6. explain where nondeterminism enters PDA design.

Why a Stack?

The First Target Language

Consider

$$A = \{0^n 1^n \mid n \geq 0\}.$$

A recognizer must remember how many 0s appeared and then verify that exactly the same number of 1s follows.

A DFA has

- finitely many states;
- no growing memory;
- no way to store arbitrary n.

A stack can

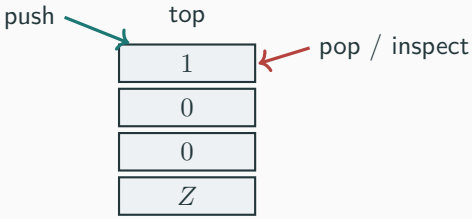
- push one marker per 0;
- pop one marker per 1;
- detect mismatch or wrong order.

A Stack Is Last-In, First-Out

A stack supports two basic operations:

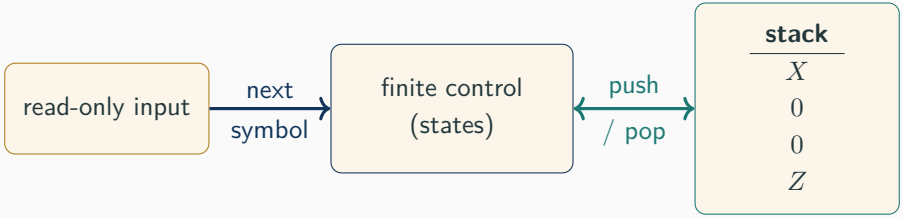
- **push:** place a symbol on top;
- **pop:** remove the top symbol.

Only the top symbol is directly accessible.



The PDA Architecture

A pushdown automaton is roughly an NFA equipped with a stack.



At each step, the PDA will use:

current state, next input symbol or ε, top stack symbol or ε.

Running Strategy for 0^n1^n

1. While reading 0s, push one 0 onto the stack for each input 0.
2. When the first 1 arrives, switch to a popping phase.
3. For every input 1, pop one stack 0.
4. Accept only when the entire input is consumed and the stack is empty.

Mental picture

The stack height records the difference

$$\#0\text{s read} - \#1\text{s read}.$$

A Trace on 000111

Unread input	Stack	Action
000111	ε	start
00111	0	read 0, push 0
0111	00	read 0, push 0
111	000	read 0, push 0
11	00	read 1, pop 0
1	0	read 1, pop 0
ε	ε	read 1, pop 0; accept

The stack is unbounded, so the same strategy works for every n.

How Bad Strings Fail

001

Input ends while a 0 remains on the stack.

011

The PDA needs to pop after the stack is already empty.

0101

After entering the 1-phase, a later 0 has no legal move.

A PDA rejects when *every possible computation branch* fails to accept.

Formal Definition

Formal Definition of a PDA

A pushdown automaton is a 6-tuple

$$P = (Q, \Sigma, \Gamma, \delta, q_0, F),$$

where:

- Q is a finite set of states;
- Σ is the input alphabet;
- Γ is the stack alphabet;
- $q_0 \in Q$ is the start state;
- $F \subseteq Q$ is the set of accept states;
- δ is the transition function.

Following Sipser's formulation,

$$\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow \mathcal{P}\big(Q \times \Gamma_\varepsilon\big),$$

where $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ and $\Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$.

Understanding the Six Components

Finite control

The set Q, the start state q_0, and the accept states F describe the finite-state part of the machine, just as in an NFA.

Input and memory

- Σ contains the symbols that may appear in the input.
- Γ contains the symbols that may be stored on the stack.

Rules of computation

The transition function δ tells the PDA:

1. which input symbol to read, if any;
2. which stack symbol to remove, if any;
3. which state to enter;
4. which stack symbol to place on top, if any.

Why Do We Need a Stack Alphabet?

The input alphabet and the stack alphabet serve different purposes.

$$\Sigma = \{\text{symbols appearing in the input}\}, \qquad \Gamma = \{\text{symbols stored as memory}\}.$$

Example: $L = \{0^n 1^n \mid n \geq 0\}$

We may use

$$\Sigma = \{0, 1\}, \qquad \Gamma = \{0, \$\}.$$

- A 0 is pushed for every 0 read from the input.
- A 0 is popped for every 1 read from the input.
- The symbol $\$$ may be used to mark the bottom of the stack.

The stack alphabet may contain:

- symbols from the input alphabet;
- completely different bookkeeping symbols;
- a special bottom-of-stack marker.

Do We Need a Bottom-of-Stack Symbol?

A special stack symbol such as

$$\$, \qquad \bot, \qquad \text{or} \qquad Z_0$$

is often placed at the bottom of the stack.

Why is it useful?

It allows the PDA to detect that all previously stored symbols have been removed.

For example, the initial move may be

$$\varepsilon, \varepsilon \to \$.$$

Later, when the PDA sees $\$$ again, it knows that the useful part of the stack is empty.

Bottom marker reached $\Longrightarrow$ all matching symbols have been popped

Important

A bottom marker is a convenient design choice, not a required component of Sipser's formal definition.

Understanding the Transition Function

Sipser defines

$$\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow \mathcal{P}(Q \times \Gamma_\varepsilon).$$

A transition examines three pieces of information:

$$\delta(\underbrace{q}_{\text{current state}}, \underbrace{a}_{\text{input action}}, \underbrace{X}_{\text{stack action}}).$$

If $(r, Y) \in \delta(q, a, X)$, then the PDA may:

1. move from state q to state r;
2. read a from the input, unless $a = \varepsilon$;
3. pop X from the stack, unless $X = \varepsilon$;
4. push Y onto the stack, unless $Y = \varepsilon$.

Thus, ε means: perform no action on that component.

Four Common Types of PDA Transitions

Suppose $(r, Y) \in \delta(q, a, X)$.

Different choices of X and Y produce different stack operations.

Pop	Push	Effect
X	Y	Replace X by Y
X	ε	Pop X
ε	Y	Push Y
ε	ε	Leave the stack unchanged

For example, $(r, \varepsilon) \in \delta(q, 1, 0)$ means:

While in q, read a 1, pop a 0, and move to r.

Similarly, $(r, 0) \in \delta(q, 0, \varepsilon)$ means:

While in q, read a 0, push a 0, and move to r.

Why Does δ Return a Set?

The codomain of the transition function is $\mathcal{P}(Q \times \Gamma_\varepsilon)$, the set of all possible transition choices.

For example,

$$\delta(q, \varepsilon, \varepsilon) = \{(q_1, \varepsilon), (q_2, \varepsilon)\}.$$

The PDA may nondeterministically choose either:

$$q \longrightarrow q_1 \qquad \text{or} \qquad q \longrightarrow q_2.$$

Why is nondeterminism useful?

A PDA may need to guess:

- where the middle of a palindrome occurs;
- which part of the input should be compared;
- which strategy or branch will lead to acceptance.

The input is accepted if *at least one* possible computation consumes the entire input and reaches an accept state.

Unpacking the Transition Function

Suppose

$$(r, c) \in \delta(q, a, b).$$

This means that the PDA may:

1. move from state q to state r;
2. read a from the input, unless $a = \varepsilon$;
3. pop b from the stack, unless $b = \varepsilon$;
4. push c onto the stack, unless $c = \varepsilon$.

Important

Because δ returns a *set* of possible moves, a PDA is generally nondeterministic.

Transition-Diagram Labels

We write a transition as

$$\boxed{\text{input},\ \text{pop} \longrightarrow \text{push}}.$$

$0, \varepsilon \to 0$

Read an input 0 and push a 0. Nothing is popped.

$\varepsilon, \varepsilon \to \$$

Read no input, pop nothing, and push the bottom marker $\$$.

$1, 0 \to \varepsilon$

Read an input 1 and pop a 0. Nothing is pushed.

$\varepsilon, \$ \to \varepsilon$

Read no input and remove the bottom marker.

The Three Roles of ε

The symbol ε may appear in three different positions.

Input is ε	Change state or stack without consuming input.
Pop symbol is ε	Do not inspect or remove the current stack top.
Push symbol is ε	Do not place a new symbol on the stack.

Common confusion

ε is an instruction meaning "do nothing here." It is not a literal symbol stored in the input or stack.

Configurations

A configuration records the complete instantaneous situation:

$$(q, w, s),$$

where

- q is the current state;
- w is the unread part of the input;
- s is the current stack content, with the top written on the left.

For example,

$$(q_{\text{pop}}, 11, 00)$$

means: the PDA is in the popping state, still has 11 to read, and has two 0s on its stack.

One Computation Step

If

$$(r, c) \in \delta(q, a, b),$$

then we may write

$$(q, aw, bs) \vdash (r, w, cs).$$

Here $a \in \Sigma_\varepsilon$ and $b, c \in \Gamma_\varepsilon$.

The notation is interpreted naturally when one of a, b, c equals ε:

- $a = \varepsilon$: no input symbol disappears;
- $b = \varepsilon$: no stack symbol disappears;
- $c = \varepsilon$: no stack symbol is added.

Acceptance by Final State

A PDA P accepts input w if there is a sequence of legal moves such that

$$(q_0, w, \varepsilon) \vdash^* (q_f, \varepsilon, s)$$

for some $q_f \in F$ and some stack content $s \in \Gamma^*$.

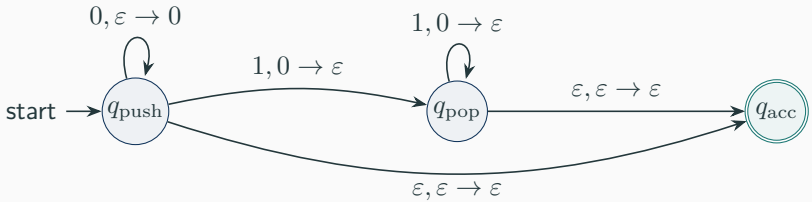
Two requirements

1. The entire input must be consumed.
2. At least one computation branch must finish in an accept state.

Acceptance by empty stack is another standard convention; it recognizes the same class of languages. We will see that later in the course.

A Complete PDA

PDA for $A = \{0^n 1^n \mid n \geq 0\}$



The direct $q_{\text{push}} \to q_{\text{acc}}$ transition handles $n = 0$, i.e., the empty string.

Why This PDA Is Correct

A correctness argument has two directions.

Every string in A is accepted

For 0^n1^n, the PDA pushes exactly n markers, then pops exactly n markers, consumes the input, and reaches q_{acc}.

Every accepted string belongs to A

An accepting computation must first read only 0s while pushing, then read only 1s while popping. Each 1 consumes exactly one earlier 0 marker, so the two counts are equal.

Formal Trace of 0011

Using configurations:

$$\begin{aligned}(q_{\text{push}}, 0011, \varepsilon) &\vdash (q_{\text{push}}, 011, 0)\\ &\vdash (q_{\text{push}}, 11, 00)\\ &\vdash (q_{\text{pop}}, 1, 0)\\ &\vdash (q_{\text{pop}}, \varepsilon, \varepsilon)\\ &\vdash (q_{\text{acc}}, \varepsilon, \varepsilon).\end{aligned}$$

Check yourself

At which step does the machine commit to the second phase?

Why Nondeterminism Matters

A New Language: Even Palindromes

Consider

$$B = \left\{ ww^R \mid w \in \{0,1\}^* \right\}.$$

Examples include

$$\varepsilon, \quad 00, \quad 11, \quad 0110, \quad 1001, \quad 010010.$$

A natural plan is:

1. push symbols from the first half;
2. at the midpoint, switch modes;
3. compare the second half against the stack.

The difficulty

There is no delimiter telling us where the midpoint occurs.

Guess the Midpoint

A nondeterministic PDA can branch at every position:

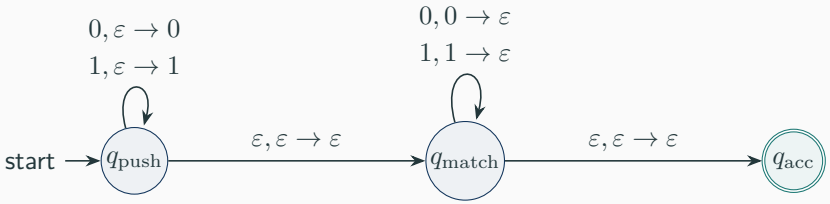
- one branch keeps pushing;
- another branch guesses, "the first half ends here," and begins popping.

Acceptance semantics

The input is accepted if *one* branch guesses the correct midpoint and all symbol comparisons succeed.

A wrong guess is harmless: that branch simply gets stuck or rejects.

PDA for $B = \{ww^R\}$



The ε-move guesses the midpoint. The matching phase verifies the reverse order.

One Accepting Branch for 0110

$$
\begin{aligned}
(q_{\text{push}}, 0110, \varepsilon) &\vdash (q_{\text{push}}, 110, 0) \\
&\vdash (q_{\text{push}}, 10, 10) \\
&\vdash (q_{\text{match}}, 10, 10) \qquad \text{(guess midpoint)} \\
&\vdash (q_{\text{match}}, 0, 0) \\
&\vdash (q_{\text{match}}, \varepsilon, \varepsilon) \\
&\vdash (q_{\text{acc}}, \varepsilon, \varepsilon).
\end{aligned}
$$

Other branches may guess too early or too late; they reject.

A Delimiter Removes the Guess

For

$$C = \left\{ w\#w^R \mid w \in \{0,1\}^* \right\},$$

the symbol $\#$ explicitly marks the midpoint.

Before $\#$	After $\#$
Push every 0 or 1.	Pop and compare every symbol.

This language can be recognized without guessing the boundary; it illustrates the difference between deterministic and nondeterministic behavior.

Another Example

Consider

$$D = \left\{0^i 1^j 2^k \mid i = j \text{ or } j = k\right\}.$$

A PDA can nondeterministically choose one of two jobs:

1. verify $i = j$ and ignore the number of 2s;
2. ignore the number of 0s and verify $j = k$.

Key principle

Nondeterministic branching naturally implements a union:

$$L_1 \cup L_2.$$

Designing PDAs

PDA Design Pattern 1: Count and Match

Use the stack as an unbounded counter when the later input must match an earlier quantity.

$$\underbrace{0\cdots0}_{\text{push}}\underbrace{1\cdots1}_{\text{pop}}$$

Typical languages:

$$\{a^n b^n \mid n \geq 0\}, \qquad \{a^i b^j c^k \mid i = j\}.$$

Caution

The stack is not a random-access counter. Its LIFO order becomes important when symbols, not just counts, must be compared.

PDA Design Pattern 2: Store and Reverse

A stack is ideal when the second part of the input must reproduce the first part in reverse order.

$$\underbrace{w}_{\text{push symbols}} \qquad \underbrace{w^R}_{\text{pop and compare}}$$

Typical languages:

$$\left\{w\#w^R\right\}, \qquad \left\{ww^R\right\}.$$

The LIFO rule automatically reverses the stored sequence.

PDA Design Pattern 3: Guess a Boundary

When a phase change is not marked in the input:

1. create an ε-transition to a new phase;
2. allow the machine to take it at any point;
3. ensure only the correct guess can complete an accepting computation.

Common uses:

- guessing a midpoint;
- guessing which condition of a union will hold;
- guessing a decomposition of the input.

A Practical Design Checklist

When constructing a PDA, ask:

1. What information must survive for an unbounded distance?
2. What exactly should each stack symbol mean?
3. When does the machine push, and when does it pop?
4. Is the phase boundary visible or must it be guessed?
5. What malformed input orders should have no transition?
6. How are ε and the empty input handled?
7. Why can no string outside the language reach an accept state?

Common Mistakes

1. Accepting as soon as the stack becomes empty, even though unread input remains.
2. Forgetting the empty string when $n = 0$ is allowed.
3. Popping a symbol without checking whether it matches.
4. Using a transition that accidentally permits symbols in the wrong order.
5. Treating nondeterminism as "try one arbitrary choice" rather than "accept if some branch succeeds."
6. Confusing ε with a real input or stack symbol.

DFA Versus PDA

	Finite Automaton	**Pushdown Automaton**
Memory	current state only	state plus unbounded stack
Access	finite control	top of stack only
Typical power	local patterns	nested / matched structure
Example	strings ending in 01	0^n1^n, palindromes
Nondeterminism	no extra language power	strictly increases power over deterministic PDAs

What One Stack Still Cannot Do

A PDA is more powerful than a finite automaton, but it is not all-powerful.

The language

$$E = \{0^n 1^n 2^n \mid n \geq 0\}$$

cannot be recognized by any one-stack PDA.

Intuition, not yet a proof

After using the stack to compare the 0s with the 1s, the machine no longer has enough independent information to compare the same n with the 2s.

A formal impossibility proof will come later.

Practice and Recap

In-Class Design Problems

Describe the stack strategy before drawing any states.

1. $L_1 = \{a^n b^{2n} \mid n \geq 0\}$
2. $L_2 = \{0^i 1^j \mid i \leq j\}$
3. $L_3 = \{w \# w^R \mid w \in \{a, b\}^*\}$
4. $L_4 = \{0^i 1^j 2^k \mid i = k\}$
5. $L_5 = \{w \in \{0, 1\}^* \mid w = w^R\}$

For each language, identify whether nondeterminism is genuinely useful.

Quick Concept Check

1. Why does δ return a set of possible moves?
2. What does the label $1, 0 \to \varepsilon$ mean?
3. Can a PDA accept with a nonempty stack under final-state acceptance?
4. Why does $\{ww^R\}$ need a guessed midpoint but $\{w\#w^R\}$ does not?
5. Why is consuming the entire input part of acceptance?

Today's Big Picture

PDA = finite-state control + one unbounded LIFO stack

Store

Push information from an earlier input region.

Compare

Pop to match counts or reverse-order symbols.

Guess

Use nondeterminism when the correct phase boundary is hidden.

Next Lecture

We will develop PDA construction more systematically and connect pushdown automata with grammars.

- richer PDA examples;
- acceptance by final state versus empty stack;
- context-free grammars;
- the equivalence between CFGs and PDAs.

A stack recognizes structure that finite memory cannot.